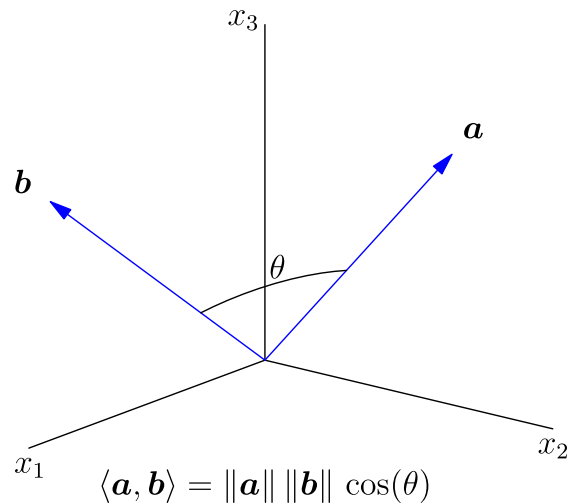


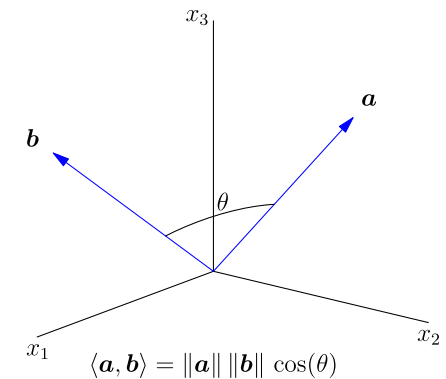


Inner products, operators

Recap

- We have looked at vector spaces (closed sets where we can add elements and multiply them by a scalar)
- Recall that vector spaces don't just apply to normal vectors ($\mathbb{R}^n$), but to matrices, functions, sequences, random variables, . . .
- Proper distances or metrics, $d(\mathbf{x}, \mathbf{y})$, allow us to construct ideas about geometry of the vector space
- Norms, $\|\mathbf{x}\|$, allow us to reason about the size of a vector
- Norms induce a distance, $d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$

1. Inner Products
2. Operators



Inner Products

- We will often consider objects with an *inner product*, $\langle, \rangle$
- For vectors in $\mathbb{R}^n$

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^\top \mathbf{y} = \sum_{i=1}^n x_i y_i$$

- For functions

$$\langle f, g \rangle = \int_{x \in \mathcal{I}} f(x) g(x) dx$$

- For $m \times n$ matrices

$$\langle \mathbf{A}, \mathbf{B} \rangle = \text{Tr} \mathbf{A}^\top \mathbf{B} = \sum_{i=1}^m \sum_{j=1}^n A_{ij} B_{ij}$$

Axioms of Inner Products

- An inner product satisfies
 1. $\langle x, x \rangle \geq 0$ for all $x \in \mathcal{V}$
 2. $\langle x, x \rangle = 0$ if and only if $x = 0$
 3. $\langle \alpha x, y \rangle = \alpha \langle x, y \rangle$
 4. $\langle x, y + z \rangle = \langle x, y \rangle + \langle x, z \rangle$
 5. $\langle x, y \rangle = \langle y, x \rangle$
- We can show that $\|x\| = \sqrt{\langle x, x \rangle}$ satisfies the axioms of a norm, so that an inner-product space is a normed space
- The norm associated with the inner-product for vectors in $\mathbb{R}^n$ (i.e. $\langle x, y \rangle = x^T y$) is the Euclidean norm $\|x\| = \sqrt{x^T x}$

Cauchy-Schwarz Inequality

- One of the most important results of inner-product spaces, known as the **Cauchy-Schwarz inequality** is that

$$\langle x, y \rangle^2 \leq \langle x, x \rangle \langle y, y \rangle = \|x\|^2 \|y\|^2$$

- Or

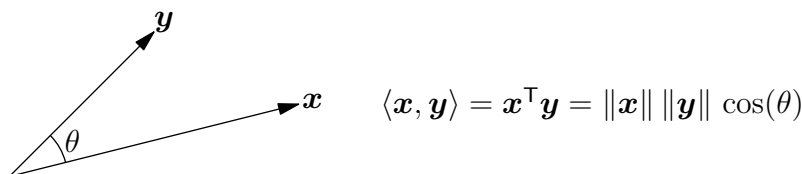
$$|\langle x, y \rangle| \leq \|x\| \|y\|$$

- This is a very general result so for example

$$\left| \int f(x) g(x) dx \right| \leq \sqrt{\left(\int f^2(x) dx \right) \left(\int g^2(x) dx \right)}$$

Angles Between Vectors

- A natural interpretation of the inner product is in providing a measure of the angle between vectors



- Vectors are orthogonal if $\langle x, y \rangle = 0$
- We can extend this idea to functions

$$\langle f(x), g(x) \rangle = \int_{x \in \mathcal{I}} f(x) g(x) dx = \|f(x)\| \|g(x)\| \cos(\theta)$$

- Note that $\sin(x)$ and $\cos(x)$ are orthogonal in the interval $[0, 2\pi]$

Basis Functions

- Any set of vectors $\{b_i | i = 1, \dots\}$ that span the space can be used as a basis or coordinate system
- The simplest and most useful case is when the vectors are orthogonal and normalised (i.e. $\|b_i\| = 1$)

- In $\mathbb{R}^3$ we could use $b_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$, $b_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$, $b_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$

- This is not unique as we can rotate our basis vectors

- For an orthonormal basis we can write any vector as

$$\hat{x} = \begin{pmatrix} x^T b_1 \\ x^T b_2 \\ x^T b_3 \end{pmatrix}$$

Orthogonal Functions

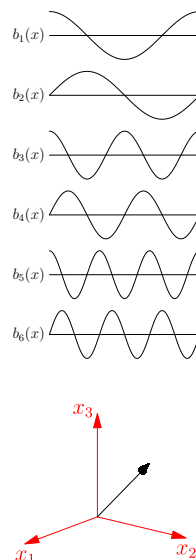
- For functions we can use any orthonormal set of functions as a basis

- The most familiar are the Fourier functions $\sin(n\theta)$ and $\cos(n\theta)$

- Any function in $C(0, 2\pi)$ can be represented by a point $\mathbf{f} = \begin{pmatrix} \langle f(x), b_0(x) \rangle \\ \langle f(x), b_1(x) \rangle \\ \vdots \end{pmatrix}$

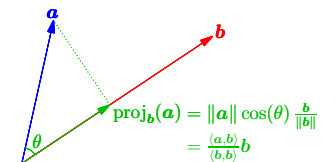
- There might be an infinite number of components

- This is analogous to points in $\mathbb{R}^n$ (for large n)



Projection Operator

- Given a vector $\mathbf{a}$ we can project it onto a vector $\mathbf{b}$



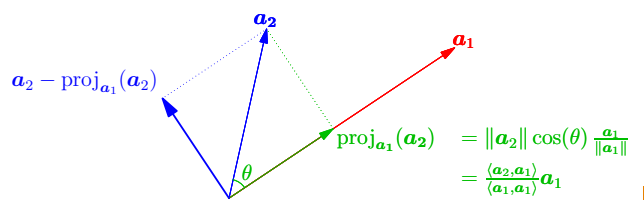
- Using $\langle \mathbf{a}, \mathbf{b} \rangle = \|\mathbf{a}\| \|\mathbf{b}\| \cos(\theta)$ and $\langle \mathbf{b}, \mathbf{b} \rangle = \|\mathbf{b}\|^2$

- A defining feature of a projection operator is

$$\begin{aligned} \text{proj}_{\mathbf{b}}(\text{proj}_{\mathbf{b}}(\mathbf{a})) &= \text{proj}_{\mathbf{b}}\left(\frac{\langle \mathbf{a}, \mathbf{b} \rangle}{\langle \mathbf{b}, \mathbf{b} \rangle} \mathbf{b}\right) \\ &= \left\langle \frac{\langle \mathbf{a}, \mathbf{b} \rangle}{\langle \mathbf{b}, \mathbf{b} \rangle} \mathbf{b}, \mathbf{b} \right\rangle \frac{\mathbf{b}}{\langle \mathbf{b}, \mathbf{b} \rangle} = \frac{\langle \mathbf{a}, \mathbf{b} \rangle}{\langle \mathbf{b}, \mathbf{b} \rangle} \mathbf{b} = \text{proj}_{\mathbf{b}}(\mathbf{a}) \end{aligned}$$

Constructing an Orthonormal Basis

- Given a set of linearly independent vectors $(\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_n)$ we can construct an orthonormal basis



- We can iterate this

$$\begin{aligned} \mathbf{u}_1 &= \mathbf{a}_1 & \mathbf{e}_1 &= \frac{\mathbf{u}_1}{\|\mathbf{u}_1\|} \\ \mathbf{u}_2 &= \mathbf{a}_2 - \text{proj}_{\mathbf{u}_1}(\mathbf{a}_2) & \mathbf{e}_2 &= \frac{\mathbf{u}_2}{\|\mathbf{u}_2\|} \\ \mathbf{u}_3 &= \mathbf{a}_3 - \text{proj}_{\mathbf{u}_1}(\mathbf{a}_3) - \text{proj}_{\mathbf{u}_2}(\mathbf{a}_3) & \mathbf{e}_3 &= \frac{\mathbf{u}_3}{\|\mathbf{u}_3\|} \end{aligned}$$

Gram-Schmidt Orthogonalisation

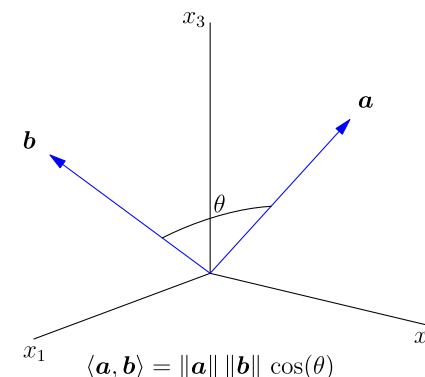
- The algorithm in the last slide is known as **Gram-Schmidt**
- We can perform it on any inner-product space
- For vectors in $\mathbb{R}^n$ it is prone to accumulate rounding errors and is not the method of choice
- Numerical packages use “Householder transformations” to perform orthogonalisation
- Nevertheless, it is nice to know that we can form an orthonormal basis

- We have gone to these lengths as we want to show that many properties of vectors are shared by other objects (matrices, functions, etc.)
- The notions of distance (geometry), norms (size of vectors) and inner products (angles between vectors) provides a very rich set of concepts
- Vectors form the backbone of objects we will use repeatedly in machine learning
- The next piece of the jigsaw is to understand how we can transform these objects

Operators

- In machine learning we are interested in transforming our input vectors into some output predictions
- To accomplish this we will apply some mapping or operators on the vector $\mathcal{T} : \mathcal{V} \rightarrow \mathcal{V}'$
- This says that $\mathcal{T}$ maps some object $x \in \mathcal{V}$ to an object $y = \mathcal{T}[x]$ in a new vector space $\mathcal{V}'$
- This new vector space may or may not be the same as the original vector space
- Our objects may be any object in a vector space such as a function

1. Inner Products
2. Operators



Linear Operators

- Operators are in general very complicated, but a particularly nice set of operators are linear operators
- $\mathcal{T}$ is a linear operator if
 1. $\mathcal{T}[ax] = a\mathcal{T}[x]$
 2. $\mathcal{T}[x + y] = \mathcal{T}[x] + \mathcal{T}[y]$
- For normal vectors ($x \in \mathbb{R}^n$) the most general linear operation is

$$\mathcal{T}[x] = \mathbf{M}x$$

where $\mathbf{M}$ is a matrix

Matrix multiplication

- For an $\ell \times m$ matrix $\mathbf{A}$ and an $m \times n$ matrix $\mathbf{B}$ we can compute the $(\ell \times n)$ product, $\mathbf{C} = \mathbf{AB}$, such that

$$C_{ij} = \sum_{k=1}^m A_{ik} B_{kj} \quad \left(\begin{pmatrix} \text{---} \\ \text{---} \\ \text{---} \end{pmatrix} \begin{pmatrix} \text{||} \\ \text{||} \\ \text{||} \end{pmatrix} = \begin{pmatrix} \text{■} & \text{■} & \text{■} \\ \text{■} & \text{■} & \text{■} \\ \text{■} & \text{■} & \text{■} \end{pmatrix} \right)$$

- Treating the vector $\mathbf{x}$ as an $n \times 1$ matrix then

$$\mathbf{y} = \mathbf{Ax} \Rightarrow y_i = \sum_j A_{ij} x_j \quad \left(\begin{pmatrix} \text{---} \\ \text{---} \\ \text{---} \end{pmatrix} \begin{pmatrix} \text{||} \\ \text{||} \\ \text{||} \end{pmatrix} = \begin{pmatrix} \text{■} \\ \text{■} \\ \text{■} \end{pmatrix} \right)$$

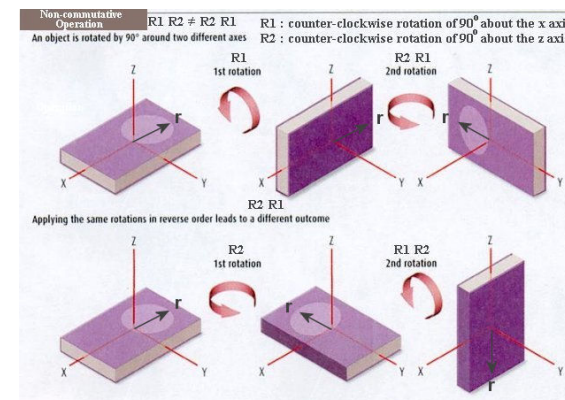
- Using the same matrix notation we can define the inner product as

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^T \mathbf{y} = \sum_{i=1}^n x_i y_i \quad \left(\begin{pmatrix} \text{---} \end{pmatrix} \begin{pmatrix} \text{||} \end{pmatrix} = \begin{pmatrix} \text{■} \end{pmatrix} \right)$$

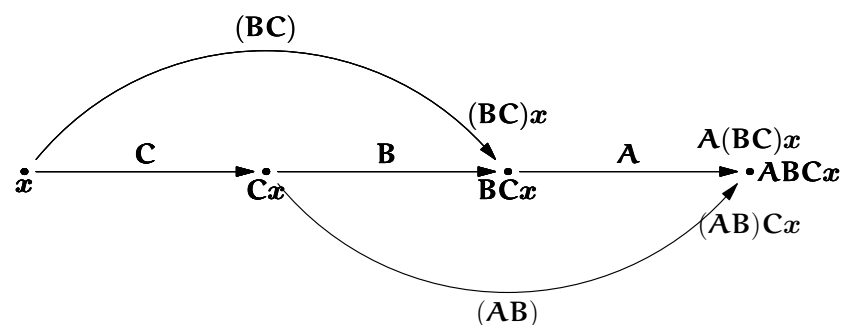
Non-commutativity

- In general $\mathbf{AB} \neq \mathbf{BA}$

$$\begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \quad \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & -1 & 0 \\ 0 & 0 & -1 \\ 1 & 0 & 0 \end{pmatrix}$$



Associativity of Mappings



- For all $\mathbf{x}$ we have $\mathbf{A(BC)x} = (\mathbf{AB})\mathbf{Cx}$
- This implies $\mathbf{A(BC) = (AB)C}$

Kernels

- The equivalent of a matrix for functions (i.e. a linear operator) is known as a kernel $K(x, y)$

$$g(x) = \mathcal{T}[f] = \int_{y \in \mathcal{I}} K(x, y) f(y) dy$$

- Our domain does not need to be one dimensional, e.g.

$$g(\mathbf{x}) = \mathcal{T}[f] = \int_{\mathbf{y} \in \mathcal{I}} K(\mathbf{x}, \mathbf{y}) f(\mathbf{y}) d\mathbf{y}$$

- We shall soon see examples of high-dimensional kernels

Kernels in Machine Learning

- Kernels are used heavily in machine learning
- In kernel methods such as SVM, SVR, Kernel-PCA
- They are also used in Gaussian Processes
- In all these cases we consider symmetric, positive semi-definite kernels
- Sometimes they can be interpreted as covariance between random functions

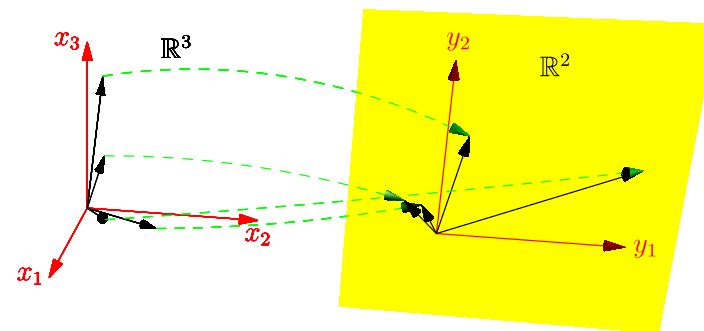
$$K(\mathbf{x}, \mathbf{y}) = \mathbb{E}_{f \sim \mathcal{P}}[(f(\mathbf{x}) - \mu(\mathbf{x}))(f(\mathbf{y}) - \mu(\mathbf{y}))]$$

Square Matrices

- We will spend a lot of time on operators that map from a vector space onto itself $\mathcal{T} : \mathcal{V} \rightarrow \mathcal{V}$
- For vectors in $\mathbb{R}^n$ such linear operators are represented by square matrices
- When there is a one-to-one mapping then we have a unique inverse
- We will study such mappings in detail in the next lecture

General Linear Mappings

- In general a linear operator will map vectors between different vector spaces
- E.g. $\mathbb{R}^3 \rightarrow \mathbb{R}^2$



Summary

- Mathematics is the language of machine learning and you have to get used to it
- Mathematics is like programming, if you don't understand the syntax and you can't write it down then it's meaningless
- We've taken a high level view of inner product spaces and operators, this will pay us back later as we look at kernel methods